FACULITY OF COMPUTING TECHNOLOGY

Department of Computer science

Individual Assignment: - Real time and embedded system

- FreeRTOS-Based LED Control System on ATmega32

Name: - Elias araya

ID: - Aku1601720

Submitted to: - Zeberhe Guesh (MSc.)

April 12, 2026

Aksum, Ethiopia

Contents

1. Introduction 2
2. System Design and Hardware Connections 2
3. FreeRTOS Integration 2
   3.1. Files Added to Project: 2
   3.2. Include Paths Added in Microchip Studio: 2
   3.3. FreeRTOSConfig.h Key Settings: 2
4. Task Creation and Priority Assignment 3
5. Observed LED Behavior 3
6. Challenges Faced 3
7. Conclusion 4
8. References 4
   Screenshots 4

1. Introduction

This report documents the design and implementation of a real-time LED control system using the ATmega32 microcontroller and FreeRTOS. The system controls three LEDs - Green, Yellow, and Red - using separate FreeRTOS tasks, each assigned a different Priority level. The project was developed using Microchip Studio and simulated using Proteus 8. The main objective was to demonstrate how FreeRTOS manages concurrent Tasks and how task priority affects the behavior of each LED.

2. System Design and Hardware Connections

The hardware consists of an ATmega32 microcontroller running at 8MHz, connected to Three LEDs through 330Ω current-limiting resistors on Port A.
Resistor value: R = (5V - 2V) / 10mA = 300Ω → rounded to 330Ω.

LED	Pin	Priority	Blink Rate
Green	PA0 (Pin 40)	3 - Highest	500ms ON / 500ms OFF
Yellow	PA1 (Pin 39)	2 - Medium	1000ms ON / 1000ms OFF
Red	PA2 (Pin 38)	1 - Lowest	2000ms ON / 2000ms OFF

The ATmega32 was configured with an 8MHz crystal oscillator connected to XTAL1 and XTAL2 pins with two 22pF load capacitors to ground. The RESET pin was connected to VCC. the AVCC pin was connected to the 5V power supply. All LED cathodes were Connected to ground.

3. FreeRTOS Integration

FreeRTOS was integrated manually into the Microchip Studio project using the official

FreeRTOS Kernel download.

3.1. Files Added to Project:

File	Purpose
tasks.c	Task creation, scheduler, delay management
list.c	Linked list used internally by the scheduler
queue.c	Queue management
port.c	AVR-specific context switching using Timer0
heap_1.c	Simple memory allocator for task stacks

3.2. Include Paths Added in Microchip Studio:

../FreeRTOS/include
../FreeRTOS/portable/GCC/ATMega323

3.3. FreeRTOSConfig.h Key Settings:

Parameter	Value	Reason
configCPU_CLOCK_HZ	8,000,000	Matches 8MHz crystal
configTICK_RATE_HZ	1000	1ms tick resolution

configTOTAL_HEAP_SIZE	1500 bytes	Fits within 2KB SRAM
configUSE_16_BIT_TICKS	1	Required for 8-bit AVR
configUSE_PREEMPTION	1	Enables priority preemption
configMAX_PRIORITIES	4	3 task levels + idle task

4. Task Creation and Priority Assignment

Three tasks were created in main() using xTaskCreate(). Each task runs an infinite Loop that toggles its LED using vTaskDelay() for timing.

```
xTaskCreate (vGreenLEDTask,  "GREEN", 100, NULL, 3, NULL);
xTaskCreate(vYellowLEDTask, "YELLOW", 100, NULL, 2, NULL);
xTaskCreate (vRedLEDTask,    "RED",   100, NULL, 1, NULL);
vTaskStartScheduler ();
```

FreeRTOS uses preemptive scheduling. When a higher-priority task becomes ready, It immediately takes the CPU. When a task calls vTaskDelay (), it enters the BLOCKED State and releases the CPU to the next highest ready task.

5. Observed LED Behavior

LED	Observed Behavior
Green	Blinked fastest - ON 500ms, OFF 500ms
Yellow	Blinked medium - ON 1000ms, OFF 1000ms
Red	Blinked slowest - ON 2000ms, OFF 2000ms

All three LEDs blinked simultaneously and independently, confirming that FreeRTOS Correctly managed three concurrent tasks. The Green task always got the CPU first When its delay expired. The Red task only ran when both Green and yellow were blocked.

6. Challenges Faced

- **Challenge 1** - FreeRTOS File Integration:

Mixing custom-written FreeRTOS files with the original downloaded kernel files caused Type conflicts and redefinition errors. Solution: use all files from the same official FreeRTOS Kernel download consistently.

- **Challenge 2** - Memory Constraints:

The ATmega32 has only 2KB of SRAM. Setting configTOTAL_HEAP_SIZE too high caused The scheduler to fail silently. Reducing it to 1500 bytes resolved the issue.

- **Challenge 3** - Include Path Configuration:

Microchip Studio required manual addition of two include paths in Toolchain settings. Missing either path caused "FreeRTOS.h: No such file or directory" errors.

- **Challenge 4 -** Proteus Simulation Setup:

Loading the correct HEX file and setting the clock frequency to exactly 8MHz was Necessary for the simulation to run correctly.

7. Conclusion

This project successfully demonstrated the use of FreeRTOS on an ATmega32 to manage Three concurrent LED tasks with different priority levels. The implementation confirmed core RTOS concepts: task creation, preemptive priority-based scheduling, and task Blocking via vTaskDelay (). The Proteus simulation provided visual confirmation that The scheduler correctly allocated CPU time based on task priority.

8. References

- FreeRTOS Official Documentation: https://www.freertos.org
- ATmega32 Datasheet: Microchip Technology Inc.
- Microchip Studio 7 User Guide

Screenshots

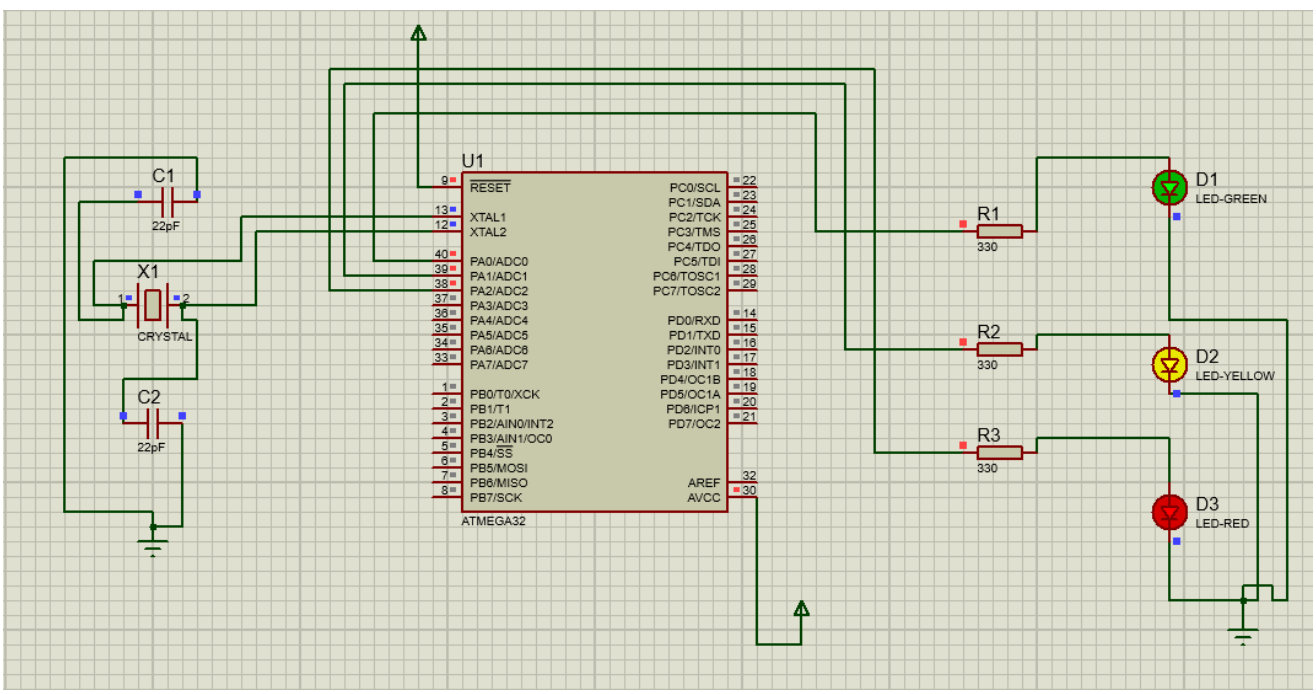

Figure 1:- simulation results from Proteus demonstrating correct operation